LithoOps AI

EUV/DUV Operations Intelligence — Project Explanation Report

Author: Orcun Kusdemir **Type:** Self-directed AI/ML learning project **Date:** July 2026

1. What this project is, and why I built it

LithoOps AI is a self-directed learning project in which I built an end-to-end operational-intelligence prototype inspired by the daily work around advanced lithography equipment (the EUV/DUV systems used in semiconductor manufacturing). My goal was to teach myself modern AI/ML engineering by solving a realistic industrial problem rather than a toy exercise: turning noisy machine sensor data into early warnings, prioritised incidents, retrieved repair knowledge, and clear shift-handover reports — always with a human approving before any action.

The operational problem it models is well documented publicly: equipment availability and fast service response are major drivers of value in this industry, and engineers contend with alarm floods, scattered technical knowledge, and incomplete shift handovers. LithoOps AI is my attempt to address those bottlenecks with a transparent, safe, and measurable system.

Scope & honesty: all data is synthetic (three simulated machines) and all maintenance documents are invented from general public knowledge. Nothing uses confidential or proprietary data. This is an independent educational project that does not belong to, and is not affiliated with, any company or entity.

2. What the system does

- **Detects anomalies early** — an Isolation Forest scores machine health from nine sensor signals and flags abnormal behaviour before it trips hard alarms.
- **Predicts failure risk and remaining useful life (RUL)** — gradient-boosted models estimate the probability of a near-term failure and how many minutes remain.
- **Triages incidents** — a six-agent team ranks urgency, identifies the likely subsystem, and always shows the supporting sensor evidence.
- **Retrieves repair knowledge (RAG)** — semantic search over a maintenance knowledge base finds the right procedure from plain-language symptoms.
- **Writes grounded shift-handover reports** — a small local language model turns the evidence and retrieved procedures into a fluent, cited summary without inventing facts.
- **Keeps humans in control** — every recommendation is read-only and must be approved by a person; every action is written to an append-only audit trail.

3. Tools and technologies used (all free / open-source)

Area	Tools	Role in the project
Language / data	Python, pandas, NumPy	Core logic and synthetic telemetry
Machine learning	scikit-learn (Isolation Forest, Gradient Boosting)	Anomaly, failure-risk, RUL models
Semantic RAG	sentence-transformers, FAISS	Meaning-based knowledge retrieval
Language model	Hugging Face Transformers (flan-t5)	Grounded report generation
Agents / tools	Custom agent team + MCP (Model Context Protocol)	Read-only, auditable tool access
Application	FastAPI, Streamlit, Plotly	REST API and control-room dashboard
Persistence	SQLite	Reference data, recommendations, audit trail
Quality / CI	pytest (19 tests), GitHub Actions	Automated verification
Compute	Local + Google Colab (free)	Heavy ML/LLM work runs free on Colab

4. Measured results

These are real numbers measured by the project's own evaluation code against known ground truth and a labelled test set — not estimates.

Capability	Metric	Result	Target
Anomaly detection	Failure-event recall	**93%**	≥ 80%
Anomaly detection	False-alert rate	**8%**	< 10%
Failure-risk model	Precision / recall	**1.00 / 1.00**	high
Remaining useful life	Mean absolute error	**~2.5 min**	low
Knowledge retrieval (RAG)	Hit@1	**0.92**	high
Knowledge retrieval (RAG)	Mean reciprocal rank	**0.92**	high
Governance	Evidence coverage	**100%**	100%
Governance	Human approval on actions	**100%**	100%

Model comparison note: I also benchmarked two language-model sizes (flan-t5-base vs flan-t5-large) on the report-writing task and found base produced more detailed but less consistent output, while large was more consistent but terser — an example of the engineering trade-offs I evaluated rather than assumed.

5. Key achievements

The results below are stated together with the real, measured metrics that support them:

• Enabled earlier fault detection on simulated lithography machines, reaching 93% failure-event recall at an 8% false-alert rate, by training an Isolation Forest health model on nine sensor signals and tuning it against the target thresholds.

• Delivered reliable maintenance-knowledge retrieval, reaching 0.92 Hit@1 and 0.92 mean reciprocal rank on a labelled test set, by building semantic search with sentence-transformer embeddings and a FAISS vector store.

• Ensured trustworthy, auditable recommendations, with 100% evidence coverage and 100% human-approval on operational actions, by designing a read-only tool layer, an agent guardrail system, and an append-only audit trail.

• Built accurate short-horizon failure prediction, with a remaining-useful-life error of about 2.5 minutes and 1.00 precision/recall on failure-risk classification, by training gradient-boosted models on labelled synthetic failure data.

6. Business framing and ownership

Beyond the models, I designed the project around the business logic of equipment operations, to show I can connect ML work to outcomes an organisation cares about:

- **Scaling:** the architecture separates lightweight serving (dashboard, API) from heavy AI compute (embeddings, LLM), which I run for free on Google Colab — the same split real teams use to scale AI cost-effectively.
- **Business outcomes:** I built an adjustable value calculator using the framework *value = incidents avoided × downtime hours avoided × value per equipment-hour – operating cost*. This demonstrates outcome thinking; the monetary figures are illustrative assumptions, clearly labelled as hypothetical rather than measured results.
- **Ownership:** I took the project from problem definition through data, modelling, retrieval, generation, evaluation, application, tests, and documentation end to end.
- **Credibility:** every capability is verified by automated tests (19 passing) and measured against explicit targets, and the honest separation of measured technical results from hypothetical business figures is itself a credibility choice.

7. What I learned

This project took me through the full modern AI/ML stack: unsupervised and supervised learning, retrieval-augmented generation, grounded language-model prompting and its failure modes (repetition, vague output), evaluation with information-retrieval metrics, agent design with guardrails, API and dashboard engineering, and the discipline of keeping a human in the loop with a full audit trail. Most importantly, I learned to **measure** what I build and to be honest about the line between demonstrated results and illustrative estimates.
